Options Tracker — Security Audit

Date: May 30, 2026
Primary concern: Prevent unauthorized brokerage access, rogue trades, money transfers

---

🔴 CRITICAL CONTEXT

The GitHub repo is **public**. This means anything ever committed — even deleted — is potentially visible in git history. This is the root of several HIGH threat items below.

Threat Matrix

#	Threat	Area	Threat Level	Fix Difficulty	Notes
1	Schwab tokens stored in Supabase `col_prefs` with anon key access	DB	**H**	M	Anyone with your Supabase URL + anon key can read `col_prefs` and extract live Schwab OAuth tokens
2	`CRON_SECRET` exposed in session summary committed to public repo	Git	**H**	L	`mYs3cr3tK3y2026` is visible in `session_summary_20260529.md` — anyone can trigger market-refresh, auto-import, DANI, auto-BTC
3	Supabase anon key in public repo / frontend bundle	Git/App	**H**	M	`VITE_SUPABASE_ANON_KEY` is baked into the frontend JS bundle served publicly — anyone can query your DB directly
4	No RLS (Row Level Security) on Supabase tables	DB	**H**	M	Anon key + no RLS = full read/write access to `contracts`, `col_prefs`, `trade_orders`, `option_snapshots` from anywhere
5	`schwab-orders` API endpoint callable with just `CRON_SECRET`	App	**H**	L	Can place real option orders if someone knows the secret (now public — see #2)
6	Schwab refresh token (7-day) stored in plaintext	DB	**H**	H	If DB is compromised, attacker gets a token that can refresh access for up to 7

#	Threat	Area	Threat Level	Fix Difficulty	Notes
	in Supabase				days
7	ETrade tokens also stored in `col_prefs` in plaintext	DB	**M**	H	Same as #6 but ETrade is currently broken so lower immediate risk
8	No rate limiting on API endpoints	App	**M**	L	`/api/auto-import`, `/api/market-refresh` etc. can be called in a loop — abuse or API quota exhaustion
9	`VITE_SUPABASE_URL` exposed in frontend	App	**L**	L	URL alone isn't dangerous but combined with anon key (#3) enables direct DB access
10	No webhook signature validation on cron triggers	App	**L**	L	Vercel cron calls aren't validated beyond the CRON_SECRET header
11	Git history may contain older secrets	Git	**M**	M	Even if rotated, old tokens/keys may exist in git history of a public repo

---

Top Priority Fixes

🔴 H-Threat / L-Difficulty — Do These Now

#2 — Rotate CRON_SECRET immediately

- `mYs3cr3tK3y2026` is in a committed markdown file in a public repo
- Go to Vercel → Environment Variables → change `CRON_SECRET` to a new random value
- Update `vercel.json` cron header if hardcoded there
- The session summary MD files should never be committed — add `session_summary_*.md` to `.gitignore`

#5 — CRON_SECRET rotation also fixes schwab-orders exposure

- Once rotated, the leaked secret can no longer trigger order placement

---

🔴 H-Threat / M-Difficulty — Do Soon

#1 + #4 — Enable Supabase RLS

- Currently anon key = full DB access
- Add RLS policies so anon key can only read/write via authenticated sessions
- For server-side API routes, switch to `SUPABASE_SERVICE_KEY` (kept only in Vercel env vars, never in frontend)
- Minimum viable RLS: restrict `col_prefs` (contains tokens!) to service role only

#3 — Separate frontend vs server Supabase keys

- Frontend only needs to read `contracts`, `col_prefs` (non-token rows) — use anon key with RLS
- Server API routes should use `SUPABASE_SERVICE_KEY` — never exposed to browser
- Rename server-side env var from `VITE_SUPABASE_ANON_KEY` to `SUPABASE_ANON_KEY` in API files so Vite doesn't bundle it

---

🟡 H-Threat / H-Difficulty — Plan for Later

#6 — Encrypt tokens at rest in Supabase

- Schwab access/refresh tokens stored as plaintext JSON in `col_prefs.cols`
- Encrypt with a server-side key before writing, decrypt on read
- Difficult because every function that reads tokens needs the decryption key

---

🟡 M-Threat / L-Difficulty — Quick Wins

#8 — Add rate limiting to API endpoints

- Vercel has built-in rate limiting (Pro plan) or use a simple in-memory counter
- At minimum: reject if same IP calls `/api/auto-import` more than 10x/minute

#11 — Audit git history for secrets

```bash
git log --all --full-history -- .env*
git grep -i "secret\|password\|token\|key" $(git rev-list --all)
```

- Use `git-secrets` or `trufflehog` to scan history
- If found: rotate all exposed values, consider `git filter-branch` to rewrite history (destructive)

What Schwab OAuth Scope Actually Allows

This is your main concern — can someone use a leaked token to transfer money?

Schwab's OAuth scopes for the Developer API are **trading only**:

- ✅ Can place option/equity orders
- ✅ Can cancel orders
- ❌ Cannot initiate ACH transfers
- ❌ Cannot wire money
- ❌ Cannot change account settings or beneficiaries

So the worst case with a stolen token is **rogue trades**, not money transfers. That's still bad (someone could buy worthless options to drain your buying power) but your brokerage funds themselves are not directly at risk of being wired out.

Recommended Action Order

1. **Today:** Rotate `CRON_SECRET` in Vercel + add `session_summary_*.md` to `.gitignore`
2. **This week:** Enable Supabase RLS on `col_prefs` table (tokens live here)
3. **This week:** Separate `VITE_` prefix from server-only env vars so anon key isn't bundled
4. **Next week:** Full RLS on all tables + service key for API routes
5. **Later:** Token encryption at rest, rate limiting, git history audit